1. What is an array?

An array stores multiple elements of the same data type under one name. Built-in C++ array elements occupy contiguous memory and are accessed using an index.

```
int arr[5] = {10, 20, 30, 40, 50};
```

- C++ arrays use zero-based indexing: the first element is arr[0].
- For an array of size n, valid indices are 0 through n-1.
- Accessing arr[n] or a negative index is out of bounds and causes undefined behaviour.
- A built-in array has fixed capacity; vector is often used when size needs to change.
- All elements in an array have the same type.

2. Declare, initialise, read and print

```
int arr[5] = {10, 20, 30, 40, 50};
cout << arr[0]; // 10
arr[2] = 99;    // update third element

int a[5];
for (int i = 0; i < 5; i++) cin >> a[i];
for (int i = 0; i < 5; i++) cout << a[i] << " ";
```

Use a loop to process elements instead of writing a separate statement for each index.

For size n, use i < n, not i <= n.

In beginner exercises, track n explicitly. sizeof(a)/sizeof(a[0]) only works for an actual array in the same scope, not after it decays to a pointer in a function parameter.

3. Sum of array elements

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
cout << sum;
```

Dry run for [2, 4, 1]

4. Find minimum and maximum

```
// Assume n >= 1
int mn = arr[0], mx = arr[0];
for (int i = 1; i < n; i++) {
    if (arr[i] < mn) mn = arr[i];
    if (arr[i] > mx) mx = arr[i];
}
```

Never access arr[0] if the array is empty. Time O(n), extra space O(1).

5. Count occurrences of a target

```
int target = 4, count = 0;
for (int i = 0; i < n; i++) {
    if (arr[i] == target) count++;
}
cout << count;
```

Use == for comparison, not =. The same pattern can count even numbers or values satisfying a condition.

6. Reverse an array using two pointers

Swap the first and last elements, then move left rightward and right leftward until they meet or cross.

```
#include <algorithm> // for swap
int left = 0, right = n - 1;
while (left < right) {
    swap(arr[left], arr[right]);
    left++;
    right--;
}
```

Step	Pointers	Array
Start	0 / 4	[1, 2, 3, 4, 5]
Swap ends	0 / 4	[5, 2, 3, 4, 1]
Move inward	1 / 3	[5, 2, 3, 4, 1]
Swap	1 / 3	[5, 4, 3, 2, 1]
Stop	2 / 2	[5, 4, 3, 2, 1]

Use left < right so the middle element doesn't need to be swapped.

Time O(n), extra space O(1).

7. Insertion and deletion by shifting

For a built-in fixed-size array, insertion is possible only if capacity is available and n tracks the current logical size. A vector is usually easier in real programs.

Insert x at index pos (capacity > n, 0 <= pos <= n)

```
for (int i = n; i > pos; i--) {
    arr[i] = arr[i - 1];
}
arr[pos] = x;
n++;
```

Delete at index pos (0 <= pos < n)

```
for (int i = pos; i < n - 1; i++) {
    arr[i] = arr[i + 1];
}
n--;
```

Insertion shifts right, so iterate backwards to avoid overwriting.
Deletion shifts left, so iterate forwards.
Worst-case time O(n) because many elements may need shifting.

Rotating an array/ vector

[1,2,3,4,5]
right= [5,4,1,2,3]

left=[3,4,5,1,2]

For right rotation by `k`, where `k` has first been reduced modulo `n`:

1. **Reverse the entire array.**
2. **Reverse the first `k` elements.**
3. **Reverse the remaining `n-k` elements.**

What if `k > n`? Use `k %= n`.

For left rotation by `k`:

1. **Reverse the first `k` elements.**
2. **Reverse the remaining elements.**
3. **Reverse the entire array.**

Difference between Array and Vector

Feature	Array	Vector
Size	Fixed	Dynamic
Insertion	Manual shifting required	Built-in methods available
Deletion	Manual shifting required	Built-in methods available
Memory	Contiguous	Contiguous

Access elements	`arr[i]`	`v[i]`
Get size	`sizeof` calculation for built-in arrays	`v.size()`
Bounds checking	No automatic checking	`v.at(i)` checks bounds
Header	No extra header needed	`#include <vector>`

```
vector<int> v = {10, 20, 30};

v.push_back(40);      // Add at the end
v.pop_back();         // Remove the last element
v.insert(v.begin(), 5); // Insert at the beginning
v.erase(v.begin());   // Remove the first element

cout << v.size();     // Number of elements
cout << v.at(0);      // Access with bounds checking
cout<<v.erase(v.begin() + 2); // deleting any element in between
```

8. Common beginner mistakes

Off-by-one errors: using i <= n when valid indices end at n-1.
Initialising sum incorrectly; for addition, start with 0.
Initialising minimum/maximum to 0, which can fail for all-negative arrays.
Reversing with two pointers but forgetting to move them.
Shifting in the wrong direction for insertion/deletion.
Forgetting that insertion into a built-in array requires spare capacity.
Assuming second-largest means distinct without checking the statement.
Ignoring n = 1, duplicates, all-negative values, or empty input.

Quiz:

What is the index of the first element?

Answer: 0

For n = 6, what is the last valid index?

Answer: 5

How many iterations does for (int i=0; i<n; i++) run?

Answer: n

What is the sum of [2, 4, 1]?

Answer: 7

Why use left < right for reversal?

Answer: Stop when pointers meet or cross.

What is the time complexity of a full traversal?

Answer: O(n)

11. Homework

https://www.geeksforgeeks.org/dsa/introduction-to-arrays-data-structure-and-algorithm-tutorials/

1. Find the sum and average of n elements.
2. Find the minimum and maximum element.
3. Count how many times a target occurs.
4. Reverse an array in place.
5. Left-rotate an array by one position.
6. Find the second-largest distinct element.